

topologyR: User Manual

Topologies of graphs, bitopologies of time series and visibility graphs (version 0.4.0)

José Mauricio Gómez Julián

Contents

1	Overview	2
2	Foundations	2
2.1	Topologies, bases and subbases	2
2.2	Finite spaces and the specialization preorder	3
2.3	Connectedness and connected components	3
3	From a graph to a topology	3
3.1	Closed neighbourhoods	3
3.2	The base, by wavefront intersection	4
3.3	Components without enumeration	4
3.4	Enumerating the topology	4
3.5	The states of a computation	4
4	Visibility graphs	5
4.1	The horizontal visibility graph	5
4.2	The natural visibility graph	5
4.3	Unequally spaced instants	7
5	Directed topologies and bitopological spaces	8
5.1	Forward and backward topologies	8
5.2	The Alexandrov topology of reachability	8
5.3	Invariants	9
5.3.1	The two base sizes are equal	9
5.3.2	Time reversal: theorem T1	10
5.3.3	What the invariants do not say	11
5.3.4	Pairwise connectedness	11
6	Implementation notes	13

7	A worked example	13
8	Legacy threshold functions	14
9	Function index	14
10	References	15

```
library(topologyR)
```

1 Overview

`topologyR` turns a graph into a topology on its vertices and computes what can be said about that topology exactly. The topology is the one generated by the *closed neighbourhoods* of the vertices: each vertex together with its neighbours. For time series, the graph is a *visibility graph*, horizontal or natural, built at equally or unequally spaced instants; its directed version, with arcs from the earlier to the later observation, yields two topologies, a forward one and a backward one, which together form a bitopological space. The package computes connected components exactly without enumerating any topology, enumerates topologies when they are small enough, and reports invariants of the forward-backward pair.

Three results organize the package and are proved in this manual:

1. In a finite space, the connected components are the classes of the equivalence generated by comparability in the specialization preorder, and that preorder can be read off the subbase. The components are therefore exact even when a computation is truncated (the section on connectedness and connected components).
2. The forward and backward bases have the same number of sets, for every digraph (the section on the equality of the two base sizes).
3. Reading a series backwards, with its instants reflected, exchanges the forward and backward topologies (theorem T1, the section on theorem T1).

The natural visibility criterion is decided exactly on the numbers the function receives, under the hypotheses on the floating-point arithmetic stated in the help page `?natural_visibility_exactness` (the section on the natural visibility graph). What the package does not do is equally important: none of its invariants is a test of the time reversibility of a stochastic process (the section on what the invariants do not say).

2 Foundations

2.1 Topologies, bases and subbases

A *topology* on a set X is a family τ of subsets of X , its *open sets*, that contains $\emptyset$ and X and is closed under arbitrary unions and finite intersections. Complements of open sets are *closed*. A family $\mathcal{B} \subseteq \tau$ is a *base* of τ when every open set is a union of members of $\mathcal{B}$. A family $\mathcal{S}$ of subsets of X *generates* the smallest topology that contains it; its open sets are the unions of finite intersections of members of $\mathcal{S}$, the empty intersection being X , and $\mathcal{S}$ is called a *subbase* of that topology. The finite intersections of a subbase form a base.

The number of sets in the base produced from a subbase is a property of the construction, not of the topology. On $\{1, 2, 3\}$, the subbases $\{\{1\}, \{2\}, \{3\}\}$ and $\{\{1, 2\}, \{2, 3\}, \{1, 3\}\}$ both generate the discrete topology, in which every subset is open, but the first is its own closure under intersections, with three sets, while the closure of the second adds the three singletons and has six.

2.2 Finite spaces and the specialization preorder

On a finite set, every family of open sets is finite, so the intersection of all open sets containing a point x is open: it is the smallest open set containing x , written U_x . If the topology is generated by a subbase $\mathcal{S}$, then $U_x = \bigcap \{s \in \mathcal{S} : x \in s\}$, with $U_x = X$ when no member contains x . A set is open if and only if it contains U_x for each of its points.

The *specialization preorder* is $x \leq y \iff y \in U_x$: every open set that contains x contains y . It can be read off any family F with $\mathcal{S} \subseteq F \subseteq \tau$: every member of F that contains x contains y if and only if $y \in U_x$. Indeed, if $y \in U_x$, every open set containing x contains U_x ; conversely, the members of $\mathcal{S}$ that contain x are among those of F , and their intersection is U_x .

2.3 Connectedness and connected components

A space is *connected* when it is not the union of two disjoint non-empty open sets; equivalently, when its only subsets that are both open and closed are $\emptyset$ and X . A *connected component* is a maximal connected subset; the components partition the space.

Proposition. *In a finite space, the connected components are the classes of the equivalence relation $\sim$ generated by comparability (x and y are comparable when $x \leq y$ or $y \leq x$).*

Proof. Each class C is open, because it contains U_x for each of its points ($y \in U_x$ means $x \leq y$), and closed, because its complement is a union of other classes; so every connected component lies inside one class. Each class is connected: if C were the union of two disjoint non-empty sets A and B , both open relative to C , some $x \in A$ and $y \in B$ would be comparable, since C is a single class; if $x \leq y$, the open set that cuts out A contains U_x and hence y , which is impossible, and the case $y \leq x$ is symmetric. $\square$

The proposition reduces connectedness to a graph search. Since the preorder can be read off any family between the subbase and the topology, so can the components, which is why `topologyR` computes them from a base that may have been truncated (the section on the states of a computation). The proposition also shows what a connected *graph* does and does not imply for the topology of its closed neighbourhoods: every closed neighbourhood lies inside one component of the graph, so a disconnected graph gives a disconnected topology, but a connected graph can give a disconnected topology.

```
path <- list(2L, c(1L, 3L), c(2L, 4L), 3L)
generate_topology(path, 4L, max_open_sets = 0L)$components
```

```
#> [[1]]
#> [1] 1 2
#>
#> [[2]]
#> [1] 3 4
```

On the path $1 - 2 - 3 - 4$, the closed neighbourhoods $\{1, 2\}$ and $\{3, 4\}$ are open and complementary, so the topology splits although the graph is connected.

3 From a graph to a topology

3.1 Closed neighbourhoods

`generate_topology()` takes adjacency lists and uses as subbase the closed neighbourhoods $N[v] = \{v\} \cup N(v)$. Nada, El Atik and Atef (2018, Definition 3.1) take as subbase the post classes of the adjacency

relation, which for a loopless graph are the open neighbourhoods $N(v)$; `topologyR` uses the closed variant, so that every vertex lies in its own subbase element, the subbase covers the vertex set, and $U_v \subseteq N[v]$. The two constructions give different topologies. On the complete graph every closed neighbourhood is the whole vertex set, and the closed variant gives the indiscrete topology:

```
complete_topology(1:6)[c("n_open_sets", "connected")]
```

```
#> $n_open_sets
#> [1] 2
#>
#> $connected
#> [1] TRUE
```

whereas the open neighbourhoods $V \setminus \{v\}$ meet in every singleton and give the discrete topology.

3.2 The base, by wavefront intersection

The base is the closure of the subbase under non-empty finite intersections. The engine computes it by wavefront propagation: each set added in a round is intersected, in the next round, with every set already in the base, until a round adds nothing. Every set added in a round is a proper subset of a set added in the previous one, so the closure ends after at most n rounds; the base itself, however, can have a number of sets exponential in n , and the argument `max_base_sets` stops the closure at the first new intersection that makes the base exceed that number.

3.3 Components without enumeration

The components are computed from the signatures $S(x) = \{B \in \mathcal{B} : x \in B\}$, stored as bitsets: x and y are comparable when $S(x) \subseteq S(y)$ or $S(y) \subseteq S(x)$, and a breadth-first search over the comparability graph returns the classes. With b base sets this takes $O(n^2 \lceil b/64 \rceil)$ word operations. Because the engine inserts the whole subbase before the closure can be cut, the signatures always encode the specialization preorder, and the components are exact.

3.4 Enumerating the topology

The open sets are enumerated, only if `max_open_sets` is positive, by the same wavefront scheme with unions. There can be up to 2^n of them, so the enumeration is meant for small spaces or for checks that need the open sets themselves. When the space is already known to be disconnected, the engine warns that the enumeration is not needed for connectivity.

3.5 The states of a computation

Base	Enumeration	base_complete	topology_complete	Components
complete	not requested	TRUE	NA	exact
complete	complete	TRUE	TRUE	exact
complete	truncated	TRUE	FALSE	exact
truncated	not requested	FALSE	NA	exact
truncated	run	FALSE	FALSE	exact

`topology_complete` is TRUE only when the enumerated family is the whole topology. A truncated base contains the subbase but not all its intersections, and the union closure of such a family is in general not the topology:

```
g <- natural_visibility_graph(c(3, 1, 4, 1, 5, 9, 2, 6))
full <- suppressWarnings(generate_topology(g$adjacency, g$n, max_open_sets = 1e5))
cut <- suppressWarnings(generate_topology(g$adjacency, g$n, max_open_sets = 1e5,
                                         max_base_sets = 12L))
c(full = full$n_open_sets, truncated_base = cut$n_open_sets)
```

```
#>          full truncated_base
#>          30             25
```

```
c(full$topology_complete, cut$topology_complete)
```

```
#> [1] TRUE FALSE
```

```
identical(full$components, cut$components)
```

```
#> [1] TRUE
```

(The warnings that the two calls suppress are the engine's notice that the space is disconnected, so that its enumeration is not needed for connectivity.)

4 Visibility graphs

4.1 The horizontal visibility graph

Two observations of a series are adjacent in the horizontal visibility graph (Luque et al., 2009) when every observation strictly between them is strictly lower than both. Consecutive observations are always adjacent. The criterion compares values and nothing else, so the graph is computed exactly on the numbers received, is invariant under any strictly increasing transformation of the values, and does not depend on the spacing of the instants.

The engine scans the series once with a stack whose values decrease strictly from bottom to top. When observation i arrives, every stacked index j with $y_j \leq y_i$ is popped and joined to i : nothing between them reaches y_j , or it would have popped j earlier. The index left on top, if any, is higher than y_i and is joined to i unless a popped value equals y_i , which blocks at height $\min = y_i$. Every other earlier index is blocked, either by that top or by the index that popped it. The algorithm runs in $O(n)$ time.

4.2 The natural visibility graph

Two observations $a < b$ are adjacent in the natural visibility graph (Lacasa et al., 2008) when every observation k between them lies strictly below the segment joining (t_a, y_a) to (t_b, y_b) , that is, when

$$D(a, b, k) = y_a(t_b - t_k) + y_b(t_k - t_a) - y_k(t_b - t_a) > 0.$$

Every horizontal edge is a natural edge, because a point lower than both ends of a segment is below it. Lacasa et al. (2008) describe the graph as inheriting properties of the series: in the words of their abstract,

periodic series convert into regular graphs, random series into random graphs, and fractal series into scale-free networks. The abstract does not say for which series these properties hold, and they do not hold for every finite series: the periodic series 0, 1, 0, 1, 0, at the equally spaced instants 1, ..., 5, has under the natural criterion the edges 1-2, 2-3, 2-4, 3-4 and 4-5 and unequal degrees,

```
lengths(natural_visibility_graph(c(0, 1, 0, 1, 0)))$adjacency)
```

```
#> [1] 1 3 2 3 1
```

The engine works by divide and conquer on the maximum. Each level of the recursion makes at most one decision per point of its intervals, so the number of decisions is $O(n^2)$ in the worst case (monotone data, whose maximum is always at an end), $O(n \log n)$ when every maximum splits its interval in parts of bounded proportion, and $O(n \log n)$ in expectation when the values are exchangeable and without ties, because the position of each maximum is then uniform, as the pivot of randomized quicksort. Three facts make the construction correct for any strictly increasing instants. (i) The maximum p of an interval blocks every pair across it, because the chord from a to b takes at t_p a convex combination of y_a and y_b , which does not exceed y_p . (ii) For $j < p$, every point between j and p is below the chord from j to p if and only if the slope from j to p is smaller than the slope from each such point to p ; scanning leftwards from p , the smallest of those slopes belongs to the last index found visible, m , so j is visible if and only if $D(j, p, m) > 0$; symmetrically on the right. (iii) The edges inside each half depend only on that half.

Exactness. Each decision is the sign of one quantity D , and the engine computes that sign exactly under the hypotheses on the arithmetic stated in the help page `?natural_visibility_exactness`. When the six inputs of a decision are at most 2^{510} in magnitude, so that no intermediate value can overflow, it evaluates in floating point the orientation determinant $O = -D$, as the difference of two rounded products l and r , and accepts the sign of the result when its magnitude exceeds $8uS$, with $u = 2^{-53}$ and S the computed value of $|l| + |r|$: above that threshold the sign is proved correct. Otherwise, near a tie, or when S does not exceed 2^{-930} , about 1.1×10^{-280} , it evaluates D exactly in integer arithmetic on the binary mantissas and exponents of its six products. This is the adaptive scheme of Shewchuk (1997). The proofs of both stages, with their hypotheses on the arithmetic (every IEEE rounding mode; evaluation in binary64, in a format with at least 64 significand bits, or in the x87 format at 53 bits) and what is checked of them, are written in the help page `?natural_visibility_exactness`; every intermediate value of the floating-point evaluation is stored, so that the compiler cannot fuse a product into the subtraction or reorder the operations. The magnitude limit is needed: a directed rounding mode can turn an overflow into the largest finite double instead of an infinity, with a relative error no longer bounded by $2u$, and a filter without the limit can accept a wrong sign. The graph is therefore the natural visibility graph of the doubles received, independent of the order in which floating-point operations round. Both constructors stop with an error when the floating-point environment does not provide IEEE 754 double arithmetic (subnormal numbers flushed to zero, or operations with fewer than 53 significant bits), a condition that some libraries create.

Decimal data. A decimal such as 0.1 has no exact binary representation, so points that are collinear in decimal can lie slightly above or below the line in binary, and the exact criterion follows the binary numbers:

```
y <- round(0.1 * (1:40) + 0.3, 1)
natural_visibility_graph(y)$n_edges
```

```
#> [1] 72
```

The 40 points lie on a line in decimal, where only consecutive points see each other, but their binary values do not. Suppose the values were recorded with d decimal places, $q_i = m_i/10^d$ with a non-negative integer $d \leq 22$ and integers $|m_i| \leq 2^{50}$, each stored as the double y_i nearest to it. Then `m <- round(10^d * y)` returns the integers m_i , and the graph of the recorded decimals is the graph of `m`, which the engine computes exactly. The check `all(abs(m) <= 2^50) && all(m / 10^d == y)` passes in that case, fails whenever some y_i is not the double nearest to a decimal with d places, and, when it passes, $m_i/10^d$ is the only decimal with d places whose nearest double is y_i :

```
m <- round(10 * y)
all(abs(m) <= 2^50) && all(m / 10 == y)
```

```
#> [1] TRUE
```

```
natural_visibility_graph(m)$n_edges
```

```
#> [1] 39
```

The proof runs in the rounding to nearest in which R computes. 10^d is an exact double for $d \leq 22$, because $5^{22} < 2^{53}$. Since y_i is nearest to q_i , $y_i = q_i(1 + \delta)$ with $|\delta| < u$; the computed product $10^d * y[i]$ equals $m_i(1 + \delta)(1 + \delta')$ with $|\delta'| < u$, so it differs from m_i by less than $2^{50}(2u + u^2)$, below $1/2$, and `round()` returns m_i ; `m[i] / 10^d`, a single correctly rounded division, returns the double nearest to q_i , which is y_i . When the check passes, y_i is the double nearest to $m_i/10^d$, and no other decimal with d places shares it, because two such decimals differ by at least 10^{-d} while two numbers rounded to the same y_i differ by at most $2u|y_i| < 10^{-d}/2$. Multiplying every value by $10^d > 0$ multiplies every D by 10^d , which keeps its sign, and the integers m_i are exact doubles. Without the bound on $|m_i|$ the recipe can fail: 2^{53} , $2^{53} + 1$ and $2^{53} + 2$ are stored as 2^{53} , 2^{53} and $2^{53} + 2$, and no rounding recovers the middle value.

4.3 Unequally spaced instants

Both constructors accept `times`, a numeric, `Date` or date-time vector of finite, strictly increasing instants of the same length as the series; `NULL` means $1, \dots, n$. The natural criterion uses the instants as abscissae; the horizontal one depends only on their order.

```
x <- c(0, 2, 4)
natural_visibility_graph(x)$edges
```

```
#>   from to
#> 1    2  3
#> 2    1  2
```

```
natural_visibility_graph(x, times = c(0, 1, 4))$edges
```

```
#>   from to
#> 1    2  3
#> 2    1  2
```

```
natural_visibility_graph(x, times = as.Date(c("2024-01-01", "2024-01-02", "2024-01-05")))$edges
```

```
#>   from to
#> 1    2  3
#> 2    1  2
```

With equally spaced instants the three points are collinear and the middle one blocks the first and third; with the instants 0, 1, 4 the middle point lies above the chord and blocks too; the dates of the third call fall 0, 1 and 4 days after the first, so they give the same graph as the instants 0, 1, 4. The reversal of this series, in the section on theorem T1, is where unequal spacing changes the answer.

5 Directed topologies and bitopological spaces

5.1 Forward and backward topologies

In the directed visibility graph every arc goes from the earlier to the later observation. The *forward topology* τ^+ is generated by the closed forward neighbourhoods $N^+[v] = \{v\} \cup \{w : v \rightarrow w\}$ and the *backward topology* τ^- by the closed backward neighbourhoods $N^-[v] = \{v\} \cup \{w : w \rightarrow v\}$. The triple (V, τ^+, τ^-) is a bitopological space, a set with two topologies, in the sense of Kelly (1963). `generate_bitopology()` builds the graph and the two topologies, together with the undirected topology and the Alexandrov topology described next.

```
series <- c(3, 1, 4, 1, 5, 9, 2, 6)
bt <- generate_bitopology(series, graph_type = "hvg")
bt$forward$components
```

```
#> [[1]]
#> [1] 1 2 3
#>
#> [[2]]
#> [1] 4 5
#>
#> [[3]]
#> [1] 6
#>
#> [[4]]
#> [1] 7 8
```

```
bt$backward$components
```

```
#> [[1]]
#> [1] 1 2
#>
#> [[2]]
#> [1] 3 4
#>
#> [[3]]
#> [1] 5
#>
#> [[4]]
#> [1] 6 7 8
```

5.2 The Alexandrov topology of reachability

The Alexandrov topology of a digraph has as open sets the sets closed under following arcs; the smallest open set containing v is the set of vertices reachable from v , v included. These reachable sets form its minimal base, which is not closed under intersection in general:

```
cross <- generate_alexandrov_topology(list(c(3L, 4L), c(3L, 4L), integer(0), integer(0)), 4L)
cross$base
```

```

#> [[1]]
#> [1] 1 3 4
#>
#> [[2]]
#> [1] 2 3 4
#>
#> [[3]]
#> [1] 3
#>
#> [[4]]
#> [1] 4

```

For the arcs $1 \rightarrow 3$, $1 \rightarrow 4$, $2 \rightarrow 3$, $2 \rightarrow 4$, the first two reachable sets meet in $\{3, 4\}$, which is open, as the union of $\{3\}$ and $\{4\}$, but is not a reachable set. The Alexandrov topology is contained in the forward topology of every digraph, since each reachable set is the union of the $N^+[w]$ of its points, but not in the undirected topology: on the path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, $\{4\}$ is open in the first and not in the second, whose smallest open set containing 4 is $\{3, 4\}$. On a visibility graph, consecutive observations always see each other, reachability is the total order, and the Alexandrov topology is the same chain $\{i, \dots, n\}$ for every series.

5.3 Invariants

`bitopology_invariants()` reports the numbers of components C^+ and C^- , their difference $D = C^- - C^+$ (`asymmetry_direction`) and $I_C = |C^+ - C^-|/\max(C^+, C^-)$ (`irreversibility_components`), the sizes of the two bases, and their excess over the Alexandrov base.

```

inv <- bt$invariants
inv[c("forward_components", "backward_components", "asymmetry_direction",
      "irreversibility_components", "forward_base_size", "backward_base_size")]

```

```

#> $forward_components
#> [1] 4
#>
#> $backward_components
#> [1] 4
#>
#> $asymmetry_direction
#> [1] 0
#>
#> $irreversibility_components
#> [1] 0
#>
#> $forward_base_size
#> [1] 11
#>
#> $backward_base_size
#> [1] 11

```

5.3.1 The two base sizes are equal

Proposition. *Let R be a relation on a finite non-empty set V , with rows $F(v) = \{w : vRw\}$ and columns $C(v) = \{w : wRv\}$. The non-empty intersections of non-empty families of rows are as many as the non-empty intersections of non-empty families of columns.*

Proof. For $A \subseteq V$ let $A' = \bigcap_{v \in A} F(v)$ (with $\emptyset' = V$), and for $B \subseteq V$ let $B^* = \{v : B \subseteq F(v)\} = \bigcap_{w \in B} C(w)$. Both maps reverse inclusion, and $A \subseteq A'^*$, $B \subseteq B'^*$; hence $A' = A'^{**}$ and $B^* = B^{**}$, and the two maps are mutually inverse bijections between the intersections of rows over all A and the intersections of columns over all B , the *intents* and *extents* of formal concept analysis (Ganter & Wille, 1999). The family in the statement is the set of intents without V , unless some row is V , and without $\emptyset$, which is an intent exactly when no column is V ; for columns the two corrections are the same two events with the roles exchanged. $\square$

The closed forward and backward neighbourhoods are the rows and columns of the reflexive closure of the arc relation, and the engine keeps exactly the non-empty intersections, so the two base sizes coincide whenever both bases are complete, and so do the two gains over the Alexandrov base, which counts the strongly connected components in either direction. The gains are counts of the construction: they are non-negative, but they do not measure how much finer τ^+ is than the Alexandrov topology, since the size of an intersection-closed base depends on the subbase that generates it.

5.3.2 Time reversal: theorem T1

Theorem (T1). *Let a series x be observed at strictly increasing instants t , let $\tilde{x}_i = x_{n+1-i}$, $\tilde{t}_i = -t_{n+1-i}$ and $r(i) = n+1-i$. Under either visibility criterion, $\{i, j\}$ is an edge for $(\tilde{x}, \tilde{t})$ if and only if $\{r(i), r(j)\}$ is an edge for (x, t) . The map r is a homeomorphism from $\tau^+(\tilde{x}, \tilde{t})$ onto $\tau^-(x, t)$ and from $\tau^-(\tilde{x}, \tilde{t})$ onto $\tau^+(x, t)$; hence C^+ and C^- are exchanged, the base sizes are exchanged, D changes sign and I_C is unchanged.*

In formulas, $C^+(\tilde{x}, \tilde{t}) = C^-(x, t)$, so $D = C^- - C^+$ changes sign and $I_C = |C^+ - C^-| / \max(C^+, C^-)$ does not change.

Proof. The horizontal criterion for $i < j$ involves only the values strictly between them, which r carries onto those strictly between $r(j)$ and $r(i)$. For the natural one, $D(r(b), r(a), r(k))$ computed for $(\tilde{x}, \tilde{t})$ equals $D(a, b, k)$ computed for (x, t) , term by term. Arcs run from the lower index to the higher, and r inverts that order, so closed forward neighbourhoods become closed backward ones; a bijection between subbases that commutes with the point map is a homeomorphism of the topologies they generate and carries bases and components onto each other. $\square$

`time_reverse()` performs the reversal. Reversing the values alone over the same instants is a different operation as soon as the gaps between instants are not palindromic, and for the natural criterion it can change the graph:

```
t <- c(0, 1, 4)
x <- c(0, 2, 4)
rv <- time_reverse(x, t)
rv
```

```
#> $series
#> [1] 4 2 0
#>
#> $times
#> [1] -4 -1 0
```

```
natural_visibility_graph(x, times = t)$edges
```

```
#>   from to
#> 1    2  3
#> 2    1  2
```

```
natural_visibility_graph(rv$series, times = rv$times)$edges
```

```
#>   from to
#> 1     1  2
#> 2     2  3
```

```
natural_visibility_graph(rev(x), times = t)$edges
```

```
#>   from to
#> 1     1  2
#> 2     1  3
#> 3     2  3
```

With the reflected instants the edge sets correspond under r ; with the same instants the reversed values make the first and third observations visible to each other, which the original series does not. Because the natural criterion is decided exactly under the hypotheses of `?natural_visibility_exactness` and negation is exact, T1 holds in the computation for every input:

```
set.seed(2026)
x <- rnorm(60)
t <- cumsum(rexp(60))
rv <- time_reverse(x, t)
d <- generate_bitopology(x, "nvg", times = t)$invariants$asymmetry_direction
d_rev <- generate_bitopology(rv$series, "nvg", times = rv$times)$invariants$asymmetry_direction
c(d, d_rev)
```

```
#> [1]  7 -7
```

5.3.3 What the invariants do not say

T1 fixes the meaning of the sign of D ; it has no converse. When a series and the gaps between its instants read the same in both directions (with equally spaced instants, a palindrome), the reversed series is the same series up to a translation of time, and $D = 0$ and $I_C = 0$ exactly. But a non-zero D does not show that the process that generated the series is irreversible, a zero D does not show that it is reversible, and the sign of D is not a prediction attached to any particular shape of the series. The probabilistic statement T1 supports concerns laws: if the joint law of the observations together with their instants is invariant under the joint reversal (values read backwards, instants reflected), the law of D is symmetric about zero, the mass at zero included; with equally spaced instants, or with fixed instants whose gaps read the same in both directions, the condition is that the law of the values equals the law of their reversal, and so it is for the horizontal criterion with any fixed instants, which it does not use. For the natural criterion with fixed instants whose gaps are not palindromic, reversing the values alone is another operation, to which the statement does not apply. None of this says anything about stationarity or about the generating process. Lacasa et al. (2012) treat time irreversibility statistically with visibility graphs, through the Kullback-Leibler divergence between the in- and out-degree distributions of the directed horizontal graph.

5.3.4 Pairwise connectedness

The bitopological space is *pairwise disconnected* when V is the union of two disjoint non-empty sets, the first open in τ^+ and the second open in τ^- . `bitopology_invariants()` decides it exactly from the two subbases, without enumerating either topology. With U_x^+ and U_x^- the smallest open sets of x , let H be the digraph

with an arc $x \rightarrow y$ when $y \in U_x^+$ or $x \in U_y^-$. A set A is open in τ^+ with its complement open in τ^- exactly when no arc of H leaves A , because $A \in \tau^+$ means $U_x^+ \subseteq A$ for $x \in A$, and $V \setminus A \in \tau^-$ means that $x \in A$ and $x \in U_y^-$ imply $y \in A$. Such a proper non-empty set exists if and only if H is not strongly connected, and a strongly connected component that no arc leaves is one. The accompanying article searches the enumerated topologies instead (its algorithm 5), with an undecided answer when an enumeration is cut; both give the same answer whenever the search decides.

On a directed visibility graph the answer is known before computing anything. Every arc goes from an earlier to a later instant, so the closed out-neighbourhood of each point lies inside the final segment that starts at it, and each final segment $\{k+1, \dots, n\}$, the union of the closed out-neighbourhoods of its points, is open in τ^+ ; in the same way each initial segment $\{1, \dots, k\}$ is open in τ^- . The two segments partition V , so the space is pairwise disconnected for every series with $n \geq 2$, and the check carries no information about the series; the witness it returns is one found by the search, valid but not necessarily the pair of segments:

```
small <- generate_bitopology(c(3, 1, 4))
small$invariants$pairwise
```

```
#> $pairwise_connected
#> [1] FALSE
#>
#> $clopen_forward
#> [1] 3
#>
#> $clopen_backward
#> [1] 1 2
```

The check can answer TRUE on digraphs with cycles. With the arcs $1 \rightarrow 2$, $2 \rightarrow 1$, $2 \rightarrow 3$, $3 \rightarrow 1$ and $3 \rightarrow 2$, the forward topology is $\{\emptyset, \{1, 2\}, V\}$ and the backward one $\{\emptyset, \{2, 3\}, V\}$; the complement of $\{1, 2\}$ is not open in the backward topology, and the space is pairwise connected although the two topologies differ:

```
out_adj <- list(2L, c(1L, 3L), c(1L, 2L))
in_adj <- list(c(2L, 3L), c(1L, 3L), 2L)
tf <- generate_topology(out_adj, 3L, max_open_sets = 100L)
tb <- generate_topology(in_adj, 3L, max_open_sets = 100L)
tf$topology
```

```
#> [[1]]
#> integer(0)
#>
#> [[2]]
#> [1] 1 2 3
#>
#> [[3]]
#> [1] 1 2
```

```
tb$topology
```

```
#> [[1]]
#> integer(0)
#>
#> [[2]]
#> [1] 1 2 3
```

```
#>
#> [[3]]
#> [1] 2 3
```

```
bitopology_invariants(tf, tb, 3L)$pairwise$pairwise_connected
```

```
#> [1] TRUE
```

6 Implementation notes

Subsets of $\{1, \dots, n\}$ are stored as bitsets of $W = \lceil n/64 \rceil$ words of 64 bits, so that intersection, union, complement and inclusion take $O(W)$ word operations. The engines are templates on W , instantiated for $W = 1, 2, 3$ ($n \leq 192$), where the compiler unrolls every loop, and a runtime version covers larger n without limit. Families of sets are deduplicated in open-addressing hash tables whose hash mixes each word with the finalizer of the generator known as splitmix64.

Step	Cost
Horizontal visibility graph	$O(n)$
Natural visibility graph	$O(n^2)$ decisions in the worst case; $O(n \log n)$ when the maxima split the intervals in parts of bounded proportion, and in expectation for exchangeable values without ties; a decision that goes to the exact evaluation costs more, by a bounded factor
Base (intersection closure)	$O(b^2 W)$ expected for a base of b sets (hashed deduplication)
Components	$O(n^2 \lceil b/64 \rceil)$
Topology (union closure)	$O(\tau ^2 W)$ expected, with $ \tau \leq 2^n$
Alexandrov reachability, index-ordered digraph	$O(nm/64)$ for m arcs
Pairwise connectedness	$O(\sum_S S ^2)$ expected over the members S of the two subbases, plus a depth-first search over the arcs of H ; at most $O(n^3)$

The costs marked as expected rest on hash tables. They assume that the hash spreads the stored sets evenly over the table, the usual uniform-hashing assumption; a fixed hash function gives no worst-case guarantee, and in the worst case a lookup takes time proportional to the number of sets stored.

7 A worked example

The following series is simulated, so that the example is reproducible; the numbers it prints describe this series and are not findings about any process.

```
set.seed(11)
s <- cumsum(rnorm(120))
for (type in c("hvg", "nvg")) {
  b <- generate_bitopology(s, graph_type = type)
  cat(type, ": edges", b$graph$n_edges,
      "| C+", b$invariants$forward_components,
      "| C-", b$invariants$backward_components,
```

```

    "| D", b$invariants$asymmetry_direction,
    "| base sizes", b$invariants$forward_base_size, b$invariants$backward_base_size,
    "| undirected components", b$invariants$undirected_info$undirected_components, "\n")
}

```

```

#> hvg : edges 217 | C+ 69 | C- 65 | D -4 | base sizes 233 233 | undirected components 50
#> nvg : edges 658 | C+ 9 | C- 11 | D 2 | base sizes 312 312 | undirected components 23

```

The two base sizes agree, as the proposition of the section on the equality of the two base sizes requires. Reversing the series changes the sign of D and leaves the rest exchanged, as T1 requires:

```

r <- time_reverse(s)
generate_bitopology(r$series, graph_type = "nvg")$invariants[c("forward_components",
                                                                "backward_components",
                                                                "asymmetry_direction")]

```

```

#> $forward_components
#> [1] 11
#>
#> $backward_components
#> [1] 9
#>
#> $asymmetry_direction
#> [1] -2

```

8 Legacy threshold functions

Version 0.1.0 built topologies from metric thresholds on the values instead of graphs, and those functions are kept:

- `calculate_thresholds()` returns five candidate radii: the mean and median gap between consecutive sorted values, the standard deviation, a quarter of the interquartile range, and an order statistic of all pairwise distances named `dbscan` for historical reasons.
- `calculate_topology()` returns the number of sets in the base generated by the neighbourhoods $\{j : |x_j - x_i| \leq h\}$, counting the empty set and the whole index set.
- `analyze_topology_factors()` and `visualize_topology_thresholds()` tabulate and plot those numbers for several radii.
- `is_topology_connected()` reports whether the graph of co-occurrence in a family of sets is connected, a necessary condition for connectedness when the family consists of open sets that cover the space; `is_topology_connected2()` is a label-dependent reachability check that is neither necessary nor sufficient; `is_topology_connected_manual()` checks coverage.
- `simplest_topology()` returns the discrete topology and `complete_topology()` the topology of the complete graph, the indiscrete one.

The radius of a threshold construction is a choice of the user; visibility graphs need none.

9 Function index

Function	Purpose
<code>horizontal_visibility_graph()</code>	horizontal visibility graph, optionally directed, with instants
<code>natural_visibility_graph()</code>	natural visibility graph, exact criterion, with instants
<code>time_reverse()</code>	a series read backwards with its instants reflected
<code>generate_topology()</code>	topology of closed neighbourhoods, exact components, enumeration
<code>generate_alexandrov_topology()</code>	Alexandrov topology of reachability of any digraph
<code>generate_bitopology()</code>	the whole pipeline for a time series
<code>bitopology_invariants()</code>	invariants of a forward-backward pair
<code>is_topology_connected_exact()</code>	connectedness of an enumerated topology
<code>calculate_thresholds()</code> , <code>calculate_topology()</code> , <code>analyze_topology_factors()</code> , <code>visualize_topology_thresholds()</code>	legacy threshold constructions
<code>is_topology_connected()</code> , <code>is_topology_connected2()</code> , <code>is_topology_connected_manual()</code>	legacy diagnostics on families of sets
<code>simplest_topology()</code> , <code>complete_topology()</code>	the discrete and the indiscrete reference topologies

Each function documents its arguments, return value and design decisions on its help page, for instance `?generate_bitopology`.

10 References

- Ganter, B., & Wille, R. (1999). *Formal concept analysis: Mathematical foundations*. Springer. <https://doi.org/10.1007/978-3-642-59830-2>
- Kelly, J. C. (1963). Bitopological spaces. *Proceedings of the London Mathematical Society*, s3-13(1), 71-89. <https://doi.org/10.1112/plms/s3-13.1.71>
- Lacasa, L., Luque, B., Ballesteros, F., Luque, J., & Nuño, J. C. (2008). From time series to complex networks: The visibility graph. *Proceedings of the National Academy of Sciences*, 105(13), 4972-4975. <https://doi.org/10.1073/pnas.0709247105>
- Lacasa, L., Nuñez, A., Roldán, É., Parrondo, J. M. R., & Luque, B. (2012). Time series irreversibility: A visibility graph approach. *The European Physical Journal B*, 85(6), Article 217. <https://doi.org/10.1140/epjb/e2012-20809-8>
- Luque, B., Lacasa, L., Ballesteros, F., & Luque, J. (2009). Horizontal visibility graphs: Exact results for random time series. *Physical Review E*, 80(4), Article 046103. <https://doi.org/10.1103/PhysRevE.80.046103>
- Nada, S., El Atik, A. E. F., & Atef, M. (2018). New types of topological structures via graphs. *Mathematical Methods in the Applied Sciences*, 41(15), 5801-5810. <https://doi.org/10.1002/mma.4726>
- Shewchuk, J. R. (1997). Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete & Computational Geometry*, 18(3), 305-363. <https://doi.org/10.1007/PL00009321>